

THE UNIVERSITY OF ZAMBIA
SCHOOL OF NATURAL AND APPLIED SCIENCES
DEPARTMENT OF COMPUTING AND INFORMATICS

FINAL YEAR PROJECT REPORT

ACADEMIC YEAR 2026

**SVDS: A Camera-Based Stolen Vehicle Detection
System for the Zambia Police Service — Real-Time
Alerts, Monitoring and System Integration**

A Final Year Project Report Submitted to the
Department of Computing and Informatics
in partial fulfilment of the requirements for the award of

[DEGREE PROGRAMME — confirm]

Student Name: [STUDENT NAME — Role 3: Real-time and Integration]

Computer Number: [COMPUTER NUMBER]

Supervisor: [SUPERVISOR NAME]

Date of Submission: [DATE OF SUBMISSION]

DECLARATION

I declare that this final year project report is my own work and has been prepared in accordance with the academic requirements of the University of Zambia. All sources of information, ideas, data, software, images, and other material that are not my own have been appropriately acknowledged and referenced.

Where this project was undertaken as part of a group, I have clearly identified my individual contributions and have not presented the work of other group members as my own.

Student Name: [STUDENT NAME — Role 3: Real-time and Integration]

Computer Number: _____

Signature: _____

Date: _____

APPROVAL

This final year project report has been submitted for examination with the approval of the project supervisor.

Supervisor Name: _____

Signature: _____

Date: _____

Departmental/Examiner **Approval** **(if** **applicable):**

ABSTRACT

Stolen-vehicle recovery in Zambia depends largely on police officers chancing to recognise a plate number from a paper report or radio bulletin. This project, carried out by a group of four students, built SVDS, a Stolen Vehicle Detection System on top of the VRRS vehicle registry. The public files stolen-vehicle reports through a web application, and a camera node reads plates from a phone's video stream and checks them against active reports, so connected police officers are alerted in real time.

This individual report covers the real-time and integration component. A WebSocket channel pushes a detection to every connected police and admin client, whichever page the officer is on, using the same token as the rest of the API. An authenticated proxy relays the phone's live video feed to the browser without exposing the phone's address. Camera-status and system-health endpoints show whether the camera and platform are working, and a start-up script and a written configuration contract let the backend, front end and camera node run as one system.

The component was tested in process with the framework's WebSocket test client, a local stand-in camera and failure-mode probes. Thirteen test cases passed: a match reached every connected client, reportee, missing and invalid tokens were refused with code 1008, a dead connection did not block the others, and the feed proxy and camera status behaved correctly for reachable and unreachable cameras. The in-process delivery time was about 17 ms, but this excludes the network, camera pipeline and browser and is not an end-to-end latency. The probes and code review found fourteen weaknesses, including that the browser never reconnects after a dropped connection, that a deactivated user's token still opens the alert channel, and that the health endpoint is unauthenticated and partly made of fixed values. None has been fixed in this project, and no live stolen-vehicle alert from a real vehicle was recorded.

Keywords: WebSocket; real-time alerts; MJPEG streaming; system monitoring; system integration

Note on the version of the code evaluated. All findings, test counts, coverage figures and line counts in this report were obtained against the code as committed (commit a349fc6). After that evaluation, uncommitted changes appeared in the working tree (middleware.py, alerts.py, system.py, main.py, api.js, SystemHealth.jsx, conftest.py, a new tests/test_regressions.py and a new front-end test). With those changes the backend suite has **38 tests, all passing**, and the front end has **one** test (npm test), which also passes. The changes appear to address the items listed below; this was established by reading the changes

and running the tests, and the original probes were **not** re-run against the changed code. Where this report says a finding was "not fixed", it refers to the evaluated commit.

Appear addressed in the working tree: **R-2** (the WebSocket re-checks the account on connect and on every message, including the heartbeat, and the feed proxy checks the account; the feed now answers a wrong role with 403 instead of 401, so TC-R9 would now expect 403), **R-9** (the ping timer is cleared on close), and the health page's database-connection card, which now shows "N/A" when the pool cannot report counts (not one of the listed findings). The changes also make the browser ignore the server's pong reply; before this, the original client would have failed to parse it as JSON, a defect this report did not identify. *Still open as far as the changes show:* R-1 (health endpoint still needs no login), R-3, R-5 to R-8, R-10 to R-13 (including the browser never reconnecting).

ACKNOWLEDGEMENTS

[Write acknowledgements here: supervisor, group members (AI/vision, backend and security, frontend), family, and anyone who helped collect plate images. The author must write this; it is personal.]

CONTENTS

Preliminary pages: Declaration · Approval · Abstract · Acknowledgements · List of Abbreviations and Acronyms · Note on Group Projects

1 Introduction — 1.1 Background and Context · 1.2 Problem Statement · 1.3 Project Aim · 1.4 Project Objectives · 1.5 Research Questions · 1.6 Scope of the Project (In Scope, Out of Scope) · 1.7 Significance · 1.8 Limitations · 1.9 Organisation of the Report

2 Literature Review — 2.1 Introduction · 2.2 Key Concepts and Theories · 2.3 Existing Systems / Related Work · 2.4 Comparative Analysis · 2.5 Research / Knowledge Gap · 2.6 Conceptual Framework · 2.7 Chapter Summary

3 Methodology — 3.1 Introduction · 3.2 Research / Development Approach · 3.3 Requirements / Data Collection · 3.4 System Architecture / Research Workflow · 3.5 Tools and Technologies · 3.6 Ethical and Legal Considerations · 3.7 Evaluation Method · 3.8 Chapter Summary

4 System Analysis and Design — 4.1 Introduction · 4.2 Stakeholder and User Analysis · 4.3 Functional Requirements · 4.4 Non-Functional Requirements · 4.5 Use Cases / User Stories · 4.6 System / Solution Architecture · 4.7 Database / Data Design · 4.8 Interface / Interaction Design · 4.9 Algorithms / Models / Technical Design · 4.10 Chapter Summary

5 Implementation / Development — 5.1 Introduction · 5.2 Development Environment · 5.3 System Components / Modules · 5.4 Key Implementation Details · 5.5 Integration · 5.6 Challenges and Solutions · 5.7 Individual Contribution · 5.8 Chapter Summary

6 Testing, Results and Evaluation — 6.1 Introduction · 6.2 Testing Strategy · 6.3 Test Cases and Results · 6.4 Experimental / Performance Results · 6.5 User Evaluation · 6.6 Analysis of Results · 6.7 Objective-by-Objective Evaluation · 6.8 Individual Results · 6.9 Chapter Summary

7 Discussion — 7.1 Introduction · 7.2 Interpretation of Findings · 7.3 Comparison with Existing Work · 7.4 Contributions of the Project · 7.5 Limitations and Implications · 7.6 Chapter Summary

8 Conclusion and Recommendations — 8.1 Introduction · 8.2 Summary of the Project · 8.3 Conclusions · 8.4 Achievement of Objectives · 8.5 Recommendations · 8.6 Future Work

References

Appendices — A Individual Contribution Statement · B User Manual / Installation Guide · C Additional System Designs / Diagrams · D Additional Test Cases and Results · E Data Collection Instruments · F Additional Code / Configuration · G Other Supporting Material

[Page numbers are not shown here. In Word, apply heading styles and use References → Table of Contents to generate a numbered contents page, and a List of Figures and List of Tables the same way.]

LIST OF ABBREVIATIONS AND ACRONYMS

Abbreviation	Meaning
ALPR	Automatic Licence Plate Recognition
API	Application Programming Interface
CUDA	Compute Unified Device Architecture (NVIDIA GPU computing platform)
RFC	Request for Comments (IETF standard document)
WS	WebSocket
DBMS	Database Management System
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IoU	Intersection over Union
JWT	JSON Web Token
mAP	mean Average Precision
MJPEG	Motion JPEG (video stream format)
ML	Machine Learning
OCR	Optical Character Recognition
SVDS	Stolen Vehicle Detection System
UI	User Interface
VRRS	Vehicle Registry and Reporting System
YOLO	You Only Look Once (object detector family)

NOTE ON GROUP PROJECTS

Students who worked on a project as a group are strongly encouraged to prepare their own individual final reports rather than submitting identical reports. The project, system, dataset, or overall research problem may be shared, but each student's report should clearly demonstrate that student's own understanding, responsibilities, work, results, and contribution to the project.

In particular, group-project students should:

- state the overall group project and explain how the individual work fits within it;
- clearly identify their own responsibilities and deliverables;
- focus detailed analysis, design, implementation, experiments, testing, and/or evaluation on their own contribution;
- acknowledge the work completed by other group members where it is discussed;
- include an *Individual Contribution Statement* in the appendices; and
- not claim another group member's work, results, code, data collection, or design decisions as their own.

Chapter 1: Introduction

1.1 Background and Context

Vehicle theft is a persistent problem for law enforcement in Zambia. Recovering a stolen vehicle depends heavily on chance: an officer happens to recognise a plate number from a paper report or a radio bulletin. Reports are kept on paper or in disconnected spreadsheets, so a report filed at one station is invisible to an officer elsewhere.

This report describes **SVDS**, the Stolen Vehicle Detection System, built by a group of four students on top of the **VRRS** registry. The public files stolen-vehicle reports through a web application. A camera node reads plates from a phone's video stream and asks the backend whether each plate belongs to an active report. When it does, connected police officers must find out *immediately*, whichever page they are looking at.

Getting that information from the moment of detection to an officer's screen, and letting the operators see whether the camera and the system are healthy, is the job of the **real-time and integration** component. It also covers the wiring that lets three separately run programs (backend, front end, camera node) behave as one system. This is the individual report of the student responsible for it.

1.2 Problem Statement

A detection is only useful if someone acts on it. If an alert reaches the police dashboard late, or not at all, the vehicle is gone. Three problems follow:

1. **Delivery.** How does a detection made by a camera process reach every logged-in officer within moments, without the officer refreshing a page, and without unauthorised users receiving it?
2. **Visibility.** How do officers and administrators know the camera is working, see what it sees, and know whether the platform itself is healthy?
3. **Integration.** How are a camera process, a web server and a browser application, developed separately, configured and started so that they work together reliably?

1.3 Project Aim

Group aim. To design, implement and evaluate SVDS: a vehicle registry and reporting web application integrated with a real-time, camera-based licence plate detection pipeline, for the

Zambia Police Service and the public it serves.

Individual aim (Real-time and integration). To build and evaluate the WebSocket alert channel, the camera status and live-feed services, the system-health endpoint, and the integration between the camera node, backend and front end.

1.4 Project Objectives

Group objectives

1. Role-based accounts (Reportee, Police, Admin) with scoped access.
2. Public filing and tracking of stolen-vehicle reports.
3. A live view of alerts, the registry and analytics for police.
4. Administrator control of accounts, system health and the audit trail.
5. Detection and reading of plates from a live camera feed, matched against active reports.
6. Real-time alerts to connected police clients.

Individual objectives (Real-time and integration)

No.	Objective	Measure of success
R1	Push a detection to all connected police and admin clients as it happens	A connected client receives a message after check-plate matches, without polling
R2	Restrict the alert channel to police and admin	Connections without a valid police or admin token are refused
R3	Report the camera's status and detection statistics	An endpoint returns online or offline and counts for the configured node
R4	Let officers watch the camera feed in the browser without exposing the camera's address	An authenticated proxy streams the feed; unauthorised requests are refused
R5	Provide a system-health endpoint for administrators	Metrics for database, disk, camera and users are returned
R6	Integrate the three processes through documented contracts and a start-up script	One command starts all three; contracts are written down
R7	Verify the real-time path and record its weaknesses honestly	Tests or probes for each objective; findings documented

1.5 Research Questions

1. Can a plain WebSocket channel, authenticated with the same token as the rest of the API, deliver detections to all connected officers with no polling?
2. Can the three processes be integrated using only network contracts, so that none needs the others' code?
3. What breaks, or could break, in this real-time design, and which of those weaknesses can be demonstrated?

(Whether the alert is *faster than the manual process it replaces* is not answered: no manual-process timing was collected, and no latency across a real network and browser was measured.)

1.6 Scope of the Project

1.6.1 In Scope

- **This report:** the WebSocket manager and endpoint, the check-plate broadcast call, camera status, the authenticated live-feed proxy, the system-health endpoint, the front end's alert connection and toast notification, the camera-status and system-health pages' use of these services, `run-all.ps1`, and the configuration that ties the phone's address into the camera node and backend.
- **Group:** the whole system.

1.6.2 Out of Scope

- The detection model and camera node (AI/vision member).
- Accounts, roles, reports, matching logic and the database (backend member).
- The visual design of the pages and the analytics charts (frontend member).
- Production hosting, TLS, and running more than one server process.

1.7 Significance of the Project

The real-time channel is what turns a passive registry into an alerting system. For the Zambia Police Service it means an officer on any page is told about a sighting as it is recorded. For the group it is the seam where three programs meet, and getting that seam right is what lets each member work independently. The report also documents where the current design would fail an officer, which matters more here than in a normal web page because a silent failure means a missed vehicle.

1.8 Limitations

- **No real network or browser measurement.** The real-time tests ran in one process using the framework's test client. The 17 ms figure reported in Chapter 6 excludes the network, the phone, video decoding, detection, OCR and the browser, and must not be read as end-to-end latency.
- **No recorded live `STOLEN` alert.** The recorded live run of the camera node (eight readings) returned `CLEAR` every time, so a real detection travelling all the way to a real dashboard has not been captured.
- **Front-end behaviour is from code reading.** The browser code (reconnection, timers, rendered text) was read, not run in a browser for this report.
- **One camera, one server process.** Nothing was tested with several cameras, several officers at scale, or several server workers.
- **Tests run on SQLite.** As in the backend report.

1.9 Organisation of the Report

Chapter 2 reviews WebSockets, streaming and monitoring concepts. Chapter 3 describes the method, tools and ethics. Chapter 4 gives the analysis and design. Chapter 5 describes the implementation and the student's contribution. Chapter 6 presents the tests, probes and objective-by-objective evaluation. Chapter 7 discusses the findings. Chapter 8 concludes. The appendices hold the contribution statement, installation guide, extra diagrams, test output and code.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the concepts behind the real-time and integration component: how servers push data to browsers, how video is streamed to a web page, how services are monitored, and how separate programs are integrated. It then reviews existing approaches and identifies the gap.

2.2 Key Concepts and Theories

2.2.1 Getting data from server to browser

HTTP is request and response: the server can only answer when the browser asks. Three techniques let a server tell a browser about something new:

- **Polling:** the browser asks repeatedly ("anything new?"). Simple, but wasteful and always late by up to one polling interval.
- **Server-Sent Events:** a one-way stream from server to browser over HTTP.
- **WebSocket (RFC 6455):** a single long-lived, two-way connection, started with an HTTP request that is "upgraded". Messages flow either way with little overhead, which suits pushing an alert the instant it exists.

2.2.2 WebSocket authentication

A browser's WebSocket object cannot set an Authorization header. Common workarounds are to put a token in the URL query string, to send it as the first message, or to use a cookie. A token in a URL is easy to implement but is likely to be written to server logs and history, and it is only checked once, when the connection is opened.

2.2.3 The publish-to-connected-clients pattern

A server keeps a list of open connections and, when an event occurs, writes the message to each. This is simple and needs no message broker, but the list lives in one process's memory. If the server runs as several processes, each has its own list, so an event handled by one process reaches only its own clients. A shared broker (for example Redis publish/subscribe) removes that limit.

2.2.4 Streaming video to a browser: MJPEG

Motion JPEG (MJPEG) sends a video as a continuous series of JPEG images in one HTTP response with the content type `multipart/x-mixed-replace`. Browsers can show it in an ordinary `<img>` tag, which is why phone "IP camera" apps use it. It is easy but uses more bandwidth than modern codecs. Because an `<img>` tag cannot send custom headers, protecting such a stream also needs a query-string token or a cookie.

2.2.5 A proxy in front of a device

A reverse proxy relays a request to another server and returns its answer. Putting the server in front of the camera means the browser never learns the camera's address, and the server can check who is asking before relaying. The costs are extra load on the server and the need to close the upstream connection when the viewer leaves.

2.2.6 Health monitoring

A health endpoint reports whether a service and its dependencies are working. Good practice is that each reported value is *measured*, not asserted, and that the endpoint is not open to the public, because it reveals internal details. Distinguishing "liveness" (the process is up) from "readiness" (its dependencies work) is common practice in operations.

2.2.7 Integration by contract

Separately deployed programs integrate by agreeing on interfaces: URLs, message shapes, ports and configuration keys. Documenting the contract, and keeping configuration in one place, reduces the chance that a change in one program silently breaks another.

2.3 Existing Systems / Related Work

- **Polling dashboards** are the simplest approach and remain common in small systems; they trade latency for simplicity.
- **Socket.IO and similar libraries** add automatic reconnection, heartbeats and fall-backs on top of WebSockets. This project uses the browser's raw WebSocket and Starlette's WebSocket support, so it has to supply these itself.
- **Message brokers (Redis, RabbitMQ, Kafka)** are the standard answer to multi-process fan-out; not used here.
- **Video platforms and surveillance systems** use RTSP, WebRTC or HLS instead of MJPEG for efficiency; MJPEG was chosen because the phone app provides it directly.
- **Monitoring tools (Prometheus, Grafana, health-check libraries)** measure service state properly; this project has a small hand-written health endpoint instead.

Note on sources. This chapter relies on standards and public documentation listed in the References. No performance figures from other systems are quoted.

2.4 Comparative Analysis

Criterion	This project	Polling	Socket.IO + broker
Delivery model	Push over WebSocket	Pull	Push
Automatic reconnection	Not implemented	Not needed	Built in
Works across several server processes	No	Yes	Yes
Setup effort	Low	Lowest	Higher
Delay to the officer	Depends on delivery only	Up to one polling interval	Depends on delivery only

The table compares design properties, not measured performance.

2.5 Research / Knowledge Gap

Tutorials show how to broadcast to WebSocket clients but seldom show what happens when a client is deactivated, a connection drops, or a slow client is present. The gap this report addresses is a working alert channel for a police setting together with a **tested account of its failure modes**.

2.6 Conceptual Framework

```
camera node --HTTP--> backend --match--> ConnectionManager --WebSocket-->
browsers (police, admin)
|
+----- authenticated MJPEG proxy -----^
(camera feed)
+----- /system/health, /system/camera-nodes ---
+ (status)
```

Detection is pushed; status is pulled; the camera feed is relayed. Each channel is authenticated with the same token as the rest of the API, except where noted in Chapter 6.

2.7 Chapter Summary

WebSockets are the right tool for pushing alerts. The simple in-memory design used here is adequate for one server process and leaves reconnection, multi-process delivery and continuous token checks to be added. MJPEG proxying and a hand-written health endpoint are workable for a prototype but need care.

Chapter 3: Methodology

3.1 Introduction

This chapter explains how the real-time and integration component was developed and evaluated: the approach, how requirements were obtained, the architecture, tools, ethical considerations and the evaluation method.

3.2 Research / Development Approach

The project was built **iteratively**. The web application and the camera-node pipeline were two work-streams that met at `POST /alerts/check-plate`. For this component the work proceeded from the seam outward: first agree what the camera sends and what the backend answers, then add the broadcast on a match, then the browser's connection and the toast, then camera status, the feed proxy and the health endpoint, and finally the start-up script.

Verification followed a **probe-and-record** approach, as in the backend report: after the features worked, the code was read for failure modes, each suspected failure was turned into a short temporary test, and only behaviour actually observed was recorded (§6.4).

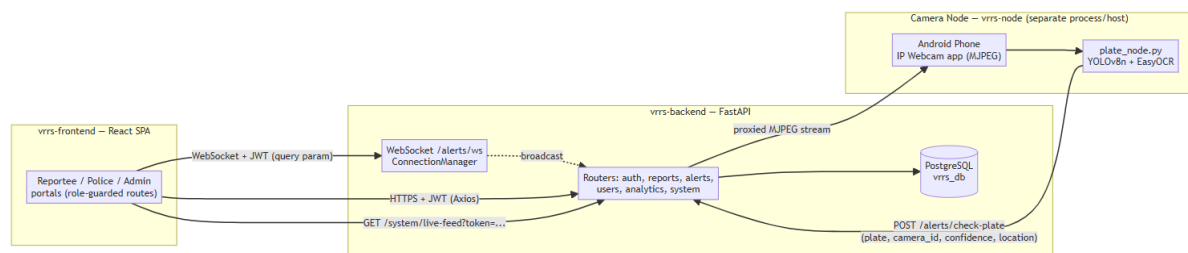
```
Requirements → Design → Implementation → Integration → Testing / Failure-mode  
review → (issues found? back)
```

3.3 Requirements / Data Collection

Requirements came from (a) the gap in Chapter 1, (b) what officers need to see (an alert wherever they are, the camera, whether the system is up), and (c) what the other members' components needed: the camera node needs a stable endpoint, and the front end needs a message shape to render. No stakeholder interviews or user studies are recorded, so the requirements are the group's own analysis.

The data flowing through this component is operational: plate strings, camera identifiers, confidence values, locations and vehicle make, model and colour from the matched report. Test data is synthetic (for example plate `RT12345`).

3.4 System Architecture / Research Workflow



Three independently run processes communicate only over the network. This component owns the arrows between them: the camera node to backend call, the backend to browser WebSocket, and the proxied camera feed.

3.5 Tools and Technologies

Area	Technology
Server-side WebSocket	FastAPI 0.111.0 / Starlette WebSocket support, served by Uvicorn 0.29.0 (uvicorn[standard])
HTTP client for camera checks and proxy	requests
Streaming	StreamingResponse relaying an MJPEG stream (multipart/x-mixed-replace)
Browser side	Native WebSocket API; React context (AlertsContext)
Camera source	Android phone running an IP Webcam app (MJPEG over Wi-Fi/hotspot)
Start-up	PowerShell script run-all.ps1
Testing	pytest, Starlette TestClient (including websocket_connect), a small local HTTP server standing in for the camera
OS	Windows 11

3.6 Ethical and Legal Considerations

- **Who receives alerts.** Alerts contain a plate, a location and the vehicle's make, model and colour. They go only to police and admin connections; reportee, missing and invalid tokens are refused (tested).
- **The camera feed.** A live feed can capture people as well as vehicles. It is limited to police and admin tokens. A real deployment would need a lawful basis, signage and a retention policy; none of these is provided by the code, and the feed is not recorded by the system.

- **Token in the URL.** Both the WebSocket and the feed take the token as a query parameter because the browser cannot send a header. Such URLs can appear in logs, so tokens can leak into log files (§6.4).
- **False alerts.** A wrong alert could send officers to an innocent driver. The alert is a lead for a human, and officers can flag false positives.
- **Health endpoint disclosure.** The health endpoint reveals disk space, database pool use, user and report counts and camera state. It is currently unauthenticated (§6.4).
- **Responsible testing.** Probes ran only against the project's own code in a test process; the local stand-in camera served only synthetic bytes. No live deployment or real camera was attacked or interfered with.

3.7 Evaluation Method

1. **Functional tests through the real WebSocket route** using the framework's test client: connect as police, cause a match, receive the message.
2. **Failure-mode probes** for authentication, disconnection handling and the health and status endpoints.
3. **A stand-in camera:** a local HTTP server that serves a short MJPEG-style stream, used to test the proxy and the online/offline logic without the real phone.
4. **Code review** of the browser code for reconnection and timers (read, not run).
5. **Objective-by-objective review** against R1 to R7 in §1.4.

Not done: measurements over a real network or with a real browser, load tests, tests with several server processes, or user evaluation.

3.8 Chapter Summary

The component was built seam-first and verified with in-process WebSocket tests, a stand-in camera and failure-mode probes. Results are about behaviour, not real-network timing.

Chapter 4: System Analysis and Design

4.1 Introduction

The system-level material (stakeholders, overall architecture) is shared by the group. The requirements, use cases, message design and monitoring design below are those of the **real-time and integration** component.

4.2 Stakeholder and User Analysis

Stakeholder	Interest	Needs from this component
Police officer	Act quickly on a sighting	An alert on any page, without refreshing; the camera view; whether the camera is up
Administrator	Keep the system running	System health; camera status
Camera operator	Keep the phone streaming	To know if the feed is reaching the server
Backend, AI/vision and frontend members	Build their parts independently	Written contracts and a single start-up command
Reportee	Nothing directly	Must not receive alerts

4.3 Functional Requirements

ID	Requirement
FR-R1	The backend keeps the set of open alert connections and, when check-plate matches, sends every connection a STOLEN_DETECTED message.
FR-R2	A connection is accepted only if it presents a valid token whose role is police or admin; otherwise it is closed with code 1008 (policy violation).
FR-R3	A client can send ping and receive pong. A connection that fails during a broadcast is dropped without affecting the others.
FR-R4	The browser opens one connection per tab after a police or admin login, keeps it across page changes, and shows a notification and an unread count when a message arrives.
FR-R5	GET /system/camera-nodes (police and admin) returns the node's id, location, online or offline status, detection count, average confidence and last-seen time.

ID	Requirement
FR-R6	GET /system/live-feed?token=... relays the camera's MJPEG stream to police and admin tokens only, returning 503 if the camera is not configured or is unreachable.
FR-R7	GET /system/health returns database, disk, camera, user and report metrics and a service list.
FR-R8	The Cameras page shows the feed and node list; the System Health page shows the health metrics.
FR-R9	A single script starts backend, front end and camera node.

4.4 Non-Functional Requirements

- **Timeliness:** an alert is sent as part of handling the matching request, with no polling delay.
- **Isolation of failure:** one dead connection must not stop delivery to others; a backend outage must not crash the camera node (the node's side).
- **Security:** only police and admin receive alerts and the feed; the camera's address is never sent to the browser.
- **Loose coupling:** the three programs share only URLs, message shapes and configuration keys.
- **Operability:** a person can tell from the interface whether the camera is reachable.

4.5 Use Cases / User Stories

UC-R1: Officer is alerted. *Precondition:* the officer is logged in on any page. The camera node reports a plate that matches an active report. The backend saves an alert and broadcasts. The officer's browser shows a "Vehicle detected" toast with the plate, location, camera and confidence, and the unread badge increases. *Alternate flows:* the officer's connection has dropped (no alert is shown; see §6.4); the officer is a reportee (no connection exists).

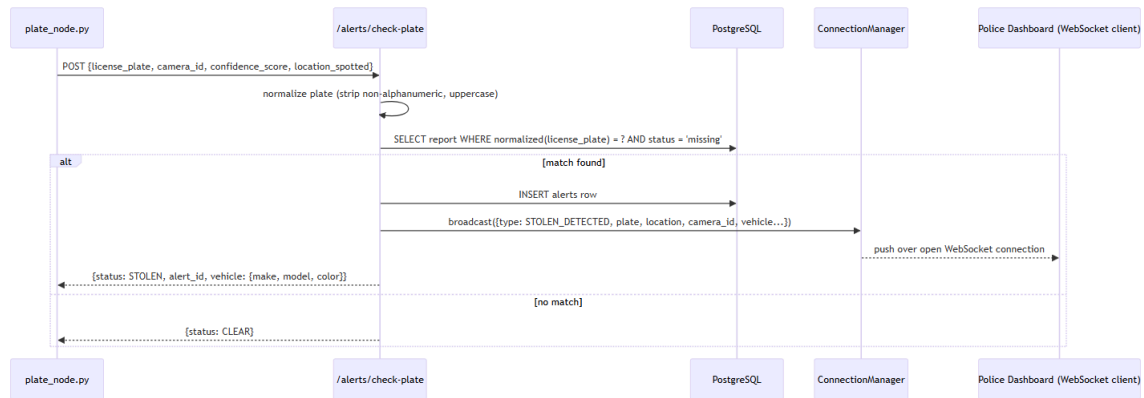
UC-R2: Officer watches the camera. On the Cameras page the browser loads the proxied feed in an image tag. If the camera is unreachable the page shows "Camera feed unavailable".

UC-R3: Administrator checks system health. The System Health page shows response time, database connections, disk, active users and camera nodes.

UC-R4: Operator starts the system. run-all.ps1 opens the backend, front end and camera node windows.

User story. *As an officer working on the analytics page, I want a stolen-vehicle sighting to appear immediately without my having to check the alerts page.*

4.6 System / Solution Architecture



Components owned here:

- **ConnectionManager** in `routers/alerts.py`: a list of open WebSockets with `connect`, `disconnect` and `broadcast`.
- **/alerts/ws**: the WebSocket route.
- **routers/system.py**: `camera-nodes`, `live-feed`, `health`, `is_camera_online()`.
- **AlertsContext** and **NotificationToast** in the front end.
- **run-all.ps1** and the two `.env` files that must agree on the camera address.

4.7 Database / Data Design

This component has no tables of its own. It **reads** the `alerts` table (count, average confidence and latest time for the configured camera) and the `users`, `reports` and `audit_log` tables (counts for the health page). The schema belongs to the backend member (Appendix C).

The alert message sent over the WebSocket is JSON:

Field	Meaning
<code>type</code>	Always <code>STOLEN_DETECTED</code>
<code>alert_id</code>	Id of the stored alert
<code>plate</code>	The plate as received, upper-cased
<code>location</code> , <code>camera_id</code> , <code>confidence</code>	As reported by the camera node
<code>vehicle_make</code> , <code>vehicle_model</code> , <code>vehicle_color</code>	From the matched report

Field	Meaning
detected_at	Server timestamp of the alert

4.8 Interface / Interaction Design

Endpoints and channels:

Interface	Direction	Authentication	Who
POST /alerts/check-plate	camera node → backend	none	public
WS /alerts/ws?token=	backend → browser	token in query string; police or admin	police, admin
GET /system/camera-nodes	browser → backend	Bearer header; police or admin	police, admin
GET /system/live-feed?token=	browser → backend → camera	token in query string; police or admin	police, admin
GET /system/health	browser → backend	none	public (see §6.4)

Interface elements (designed with the frontend member): a toast at the top of every page, an unread badge in the police navigation, the Cameras page (feed and node table) and the System Health page (four metric cards, core services, camera nodes).

Configuration contract:

Setting	Where	Must match
Phone stream address	vrrs-node/.env PHONE_STREAM_URL	Backend .env CAMERA_STREAM_URL
Camera id	vrrs-node/.env CAMERA_ID	Backend .env CAMERA_NODE_ID
Backend address	vrrs-node/.env BACKEND_URL	The port the backend runs on
API and WebSocket addresses	vrrs-frontend/.env VITE_API_BASE_URL, VITE_WS_URL	The backend

4.9 Algorithms / Models / Technical Design

4.9.1 Broadcast

```
on match: save alert -> for each connection: try send(json) ; collect failures  
-> remove failed
```

Sending is sequential: one slow client delays the ones after it. Failed connections are removed after the loop.

4.9.2 Handshake authentication

```
connect(token) -> decode_token(token) -> role in {police, admin}? -> accept :  
close(1008)
```

The token is checked once. After acceptance the connection is trusted until it closes.

4.9.3 Camera online check

`is_camera_online()` performs an HTTP GET on the stream address with a 3-second timeout and returns true if the connection opens. It is called whenever camera status or health is requested.

4.9.4 Feed proxy

```
check token role -> open upstream GET (stream, 10 s connect timeout) -> stream 4  
KB chunks to the browser
```

4.9.5 Health metrics

Values are computed at request time: database round trip (`SELECT 1`), disk usage of the drive, connection-pool counts, counts from tables. Some fields are fixed values, discussed in §6.4.

4.10 Chapter Summary

The design uses a simple in-memory connection list, a one-time token check at connect, a relayed MJPEG feed and a hand-written health endpoint. Contracts and configuration keys are written down so the three programs integrate without sharing code.

Chapter 5: Implementation / Development

5.1 Introduction

This chapter describes how the real-time and integration component was built, how it connects to the other members' work, the problems met, and the student's contribution.

5.2 Development Environment

Item	Detail
Machine	Windows 11 PC
Backend runtime	Python with FastAPI 0.111.0, Uvicorn 0.29.0 (uvicorn[standard] provides WebSocket support), requests
Front end	React with Vite (dev server on port 5173)
Camera	Android phone with an IP Webcam app, streaming at <code>http://<phone-ip>:8080/video</code>
Tools	Swagger UI for HTTP routes, pytest for tests, PowerShell for start-up

5.3 System Components / Modules

Module	Purpose	Built by
ConnectionManager and <code>/alerts/ws</code> in <code>app/routers/alerts.py</code>	Track connections, authenticate the handshake, broadcast	Real-time (this report)
The <code>manager.broadcast(...)</code> call inside <code>check-plate</code>	Trigger the broadcast after a match	Shared: the endpoint is the backend member's, the call is agreed with this member
<code>app/routers/system.py</code> : <code>camera-nodes</code> , <code>live-feed</code> , <code>health</code> , <code>is_camera_online</code>	Camera status, feed proxy, health	Real-time (this report)
<code>/system/audit</code> in the same file	Audit-log listing	Backend/security member's records; listing route shared
<code>src/context/AlertsContext.jsx</code> , <code>components/NotificationToast.jsx</code> ,	Browser connection, unread	Real-time (this report)

Module	Purpose	Built by
createAlertSocket in services/api.js	count, toast	
pages/police/Cameras.jsx, pages/admin/SystemHealth.jsx	Consume status, feed and health	Layout by frontend member; data wiring here
run-all.ps1 and .env keys	Start-up and configuration	Real-time (this report)

5.4 Key Implementation Details

5.4.1 Connection manager and WebSocket route

```
class ConnectionManager:
    def __init__(self): self.active_connections = []
    async def connect(self, ws): await ws.accept();
self.active_connections.append(ws)
    def disconnect(self, ws): self.active_connections.remove(ws)
    async def broadcast(self, data):
        message = json.dumps(data); disconnected = []
        for c in self.active_connections:
            try: await c.send_text(message)
            except Exception: disconnected.append(c)
        for c in disconnected: self.active_connections.remove(c)

@router.websocket("/ws")
async def websocket_endpoint(websocket):
    token = websocket.query_params.get("token")
    payload = decode_token(token) if token else None
    if not payload or payload.get("role") not in ("police", "admin"):
        await websocket.close(code=1008); return
    await manager.connect(websocket)
    try:
        while True:
            if await websocket.receive_text() == "ping": await
websocket.send_text("pong")
            except WebSocketDisconnect: manager.disconnect(websocket)
```

(Shown in condensed form.) The route reuses `decode_token` from the backend member's `auth.py`, so the same tokens work for REST and WebSocket.

5.4.2 Browser side

AlertsProvider opens the socket once when the user's role is police or admin, and closes it when the provider unmounts:

```
const ws = createAlertSocket((data) => {
  if (data.type === "STOLEN_DETECTED") {
    setUnread(prev => prev + 1); setNotification(data); setLastEvent(data);
  }
});
```

NotificationToast is mounted once at the top of App, so it shows on every page. Alerts.jsx prepends lastEvent to its list when the "unread" view is open, so the new alert appears without a refresh. createAlertSocket builds the URL `ws://<host>/alerts/ws?token=<jwt>` from VITE_WS_URL, sends ping every 30 seconds, and logs close and error events to the console.

5.4.3 Camera status and health

camera-nodes combines `is_camera_online()` with a database query for the configured camera's detection count, average confidence and latest detection time. Old readings stored as fractions (0 to 1) are scaled to percentages with the same expression used in the analytics code. health returns metrics and a fixed list of four services. Its database timing is a `SELECT 1` round trip, its disk figures come from `shutil.disk_usage`, and its pool figures from the SQLAlchemy engine pool.

5.4.4 The feed proxy

```
payload = decode_token(token)
if not payload or payload.get("role") not in ("police", "admin"): raise
HTTPException(401)
upstream = requests.get(CAMERA_STREAM_URL, stream=True, timeout=10)
return StreamingResponse(upstream.iter_content(chunk_size=4096),
media_type=content_type)
```

On the front end the Cameras page sets an image tag's src to `/system/live-feed?token=...`, and shows a message if the image fails to load.

5.4.5 Start-up script

run-all.ps1 opens three PowerShell windows: the backend (uvicorn on port 8000, using the project's virtual environment), the front end (npm run dev), and the camera node (the GPU

Python environment running `plate_node.py`). It prints a reminder to update the phone's address first.

5.5 Integration

- **With the camera node (AI/vision member):** the node calls `check-plate`; it needs no knowledge of WebSockets. The camera id it sends must equal the backend's `CAMERA_NODE_ID` for the Cameras page statistics to count its detections.
- **With the backend (backend member):** reuses the token functions and the database session; adds routes under `/alerts` and `/system`.
- **With the front end (frontend member):** the message shape of §4.7 is the contract; the toast and the alerts list read `plate`, `location`, `camera_id`, `confidence`. The live message has no `is_read` field and calls its identifier `alert_id`, while stored alerts use `id`; the Alerts page handles both spellings.
- **Configuration:** the phone's address must be entered twice, once in each of two `.env` files. On the development machine both currently hold the same address (checked).

5.6 Challenges and Solutions

Challenge	Cause	Solution
Browser cannot send an Authorization header on a WebSocket or an <code></code> request	Browser API limits	Token passed as a query parameter and checked in the handler (a known trade-off, §6.4)
Exposing the camera's raw address to browsers	The phone app has no authentication of its own	Relay through an authenticated proxy so only the server knows the address
Officer on a non-alert page would miss a detection	Per-page connections would only exist on the alerts page	One connection in a top-level context plus a top-level toast
A dead connection should not stop delivery	Sending to a closed socket raises an error	Broadcast catches per-connection errors and removes failed connections afterwards
Three programs to start in the right way	Separate runtimes and environments	<code>run-all.ps1</code>
Phone address changes on reconnect	DHCP on the phone's hotspot	Documented manual edit of two <code>.env</code> files; automatic discovery is future work

Challenge	Cause	Solution
Camera unreachable during health checks	Blocking network call	3-second timeout; measured at about 2 s to report offline (§6.4)

5.7 Individual Contribution (group project)

The student was responsible for **work area 5** of the group's seven (real-time alerts and monitoring), together with the **integration** of the phone, camera node, backend and front end (Appendix A). This covers:

- the WebSocket connection manager, route and handshake check;
- the broadcast message design and the call from check-plate;
- camera status, the authenticated live-feed proxy and the health endpoint;
- the browser's alert connection, unread count and toast;
- the configuration contract and `run-all.ps1`;
- the failure-mode review and the tests and probes in Chapter 6.

Work by others that this component uses, **not claimed here**: the token, roles and check-plate matching (backend member); the detection model and camera node (AI/vision member); the page layouts, styling and analytics (frontend member). The backend tests in `vrrs-backend/tests/` include some for alerts; who wrote them should be confirmed by the student.

5.8 Chapter Summary

The component is a small in-memory WebSocket broadcaster, a relayed MJPEG feed, a status and health service, and the browser code and scripts that connect them. Its main design decisions are one shared connection per tab, per-connection error isolation, and tokens in the query string.

Chapter 6: Testing, Results and Evaluation

6.1 Introduction

This chapter evaluates the real-time and integration component. Results come from temporary test files that were written for this report, run in the project's test environment, and then **deleted**, so the shipped test suite is unchanged (31 tests, still passing). Their printed output is reproduced in Appendix D.

6.1.1 Version of the code evaluated

Note on the version of the code evaluated. All findings, test counts, coverage figures and line counts in this report were obtained against the code as committed (commit a349fc6). After that evaluation, uncommitted changes appeared in the working tree (middleware.py, alerts.py, system.py, main.py, api.js, SystemHealth.jsx, conftest.py, a new tests/test_regressions.py and a new front-end test). With those changes the backend suite has **38 tests, all passing**, and the front end has **one** test (npm test), which also passes. The changes appear to address the items listed below; this was established by reading the changes and running the tests, and the original probes were **not** re-run against the changed code. Where this report says a finding was "not fixed", it refers to the evaluated commit.

Appear addressed in the working tree: **R-2** (the WebSocket re-checks the account on connect and on every message, including the heartbeat, and the feed proxy checks the account; the feed now answers a wrong role with 403 instead of 401, so TC-R9 would now expect 403), **R-9** (the ping timer is cleared on close), and the health page's database-connection card, which now shows "N/A" when the pool cannot report counts (not one of the listed findings). The changes also make the browser ignore the server's pong reply; before this, the original client would have failed to parse it as JSON, a defect this report did not identify. *Still open as far as the changes show:* R-1 (health endpoint still needs no login), R-3, R-5 to R-8, R-10 to R-13 (including the browser never reconnecting).

6.2 Testing Strategy

Level	Technique	Target
Functional (in process)	Starlette TestClient.websocket_connect against the real route	Handshake, broadcast, ping, rejection
Failure-mode probes	Direct calls to the manager; token states	Disconnect handling, dead connections, deactivated users

Level	Technique	Target
Service tests with a stand-in camera	A local HTTP server serving a short MJPEG-style stream	Feed proxy, online/offline status, 503 behaviour
Endpoint inspection	Calls without and with tokens	Health and status endpoints
Code review	Reading the browser code	Reconnection, timers, rendered fields
Acceptance / user testing	None	See §6.5

Limits of the environment. The server and the client run in one process, on an in-memory SQLite database. The stand-in camera is a local server, not the phone. Nothing here involves a real network, a real browser, video decoding or the detector. Front-end results in §6.4.3 come from reading code, not running it.

6.3 Test Cases and Results

Table 6.1: Test cases

ID	Test case	Expected result	Observed	Status
TC-R1	Police client connected; check-plate matches an active report	Client receives STOLEN_DETECTED with plate and vehicle make	Received STOLEN_DETECTED, plate RT12345, make Toyota; HTTP answer STOLEN	Pass
TC-R2	Client sends ping	pong	pong	Pass
TC-R3	One police and one admin client connected; one match	Both receive it	Both received	Pass
TC-R4	Connect with a reportee token	Refused	Closed with code 1008	Pass
TC-R5	Connect with no token	Refused	Closed with code 1008	Pass
TC-R6	Connect with an invalid token	Refused	Closed with code 1008	Pass
TC-R7	Broadcast when one connection is dead	Dead one dropped; live one still receives	One connection remained; live one received the message	Pass

ID	Test case	Expected result	Observed	Status
TC-R8	Live-feed proxy with a reachable stand-in camera and a police token	200 and the stream's frames	200, multipart/x-mixed-replace, 132 bytes including the frame data	Pass
TC-R9	Live feed with a reportee token	Refused	401	Pass
TC-R10	Live feed with no token	Refused	422 (missing required parameter)	Pass
TC-R11	Camera status with the stand-in camera reachable	online	online; health reported one node online	Pass
TC-R12	Camera unreachable	Feed 503; status offline	503 after about 2.0 s; offline after about 2.1 s	Pass
TC-R13	Camera status with no stream configured or reachable	offline	offline; the node list has the fields id, location, status, detections, average confidence, last seen	Pass

6.4 Results and Findings

6.4.1 Delivery

In the in-process test, the time from calling check-plate to the client receiving the message was **about 17 ms** (one run: 16.9 ms). This measures the framework and the database write only. It excludes the network, the phone, video decoding, detection, OCR, the camera node's frame skipping and the browser's rendering, so it says nothing about the time an officer would experience. **No end-to-end latency was measured**, and it would depend mostly on the camera node's steps, not on the alert channel.

Delivery to several clients worked (TC-R3). Delivery is sequential: the loop awaits each client in turn, so a slow client delays those after it. This was not tested with a slow client.

6.4.2 Findings from the probes

Table 6.2: Findings

ID	Finding	Evidence	Severity (student's judgement)
R-1	GET /system/health needs no login. It returns disk usage, free disk space, database pool use, user and report counts, and camera status	Called without a token: 200 with the metrics	Medium
R-2	A deactivated user's token still opens the alert channel (the token is checked only at connect, with no database check)	Token of a user set inactive: connection accepted and ping answered pong	High. Same root cause as the backend report's F-1
R-3	disconnect() raises if the connection was already removed	Calling it twice raised <code>ValueError: list.remove(x): x not in list</code> . A sequence that would cause this is a connection removed by a failed broadcast that then disconnects normally; that sequence was inferred, not reproduced	Low
R-4	Health values are partly fixed or misnamed. "active sessions" counts active <i>accounts</i> , not sessions (it returned 0 with no users). "API response time" is the time of one <code>SELECT 1</code> . The number of camera nodes is fixed at 1. Three of four services (backend, WebSocket, JWT) are always reported "operational", whatever their state	Probe output and code	Medium: an administrator can be reassured falsely
R-5	Camera nodes response lacks last_ping_seconds , which the System Health page reads to show "Last ping: ... ago"	The response keys did not include it. From the page's code, an online node would show the text "Last ping: undefineds ago". Not confirmed in a browser	Low
R-6	Camera status is a full HTTP connection each time , blocking for up to 3 s when the camera is unreachable	About 2.1 s to report offline in the test; the System Health page calls both <code>health</code> and <code>camera-nodes</code> , so it probes twice	Low
R-7	The token appears in the URL for both the WebSocket and the feed	Read from the code and the client	Medium (leak into logs and history)

6.4.3 Findings from code reading (not run)

- **R-8 No reconnection in the browser (High).** `createAlertSocket` handles close and error only by writing to the console. If the connection drops (network change, laptop sleep, server restart, or token expiry), the officer gets **no alerts and no on-screen sign** until they reload the page. The unread count is fetched once at start. For an alerting system this is the most serious real-time weakness.
- **R-9 Timer leak.** Each socket starts a 30-second ping timer that is never cleared; after a close it keeps running, and after a re-login another one is started.
- **R-10 One notification slot.** The toast holds a single notification and has no automatic dismiss; a second detection replaces the first one on screen. (The alert is still added to the alerts list when that page is open.)
- **R-11 In-memory connection list.** The list lives in one server process. With several worker processes, an alert would reach only clients connected to the worker that handled the matching request. Not tested.
- **R-12 Feed proxy resources.** The relay holds a worker thread for as long as a viewer watches, opens a separate upstream connection for each viewer (so several officers watching multiply the load on the phone), and does not explicitly close the upstream connection when the viewer leaves. Not measured.
- **R-13 Configuration duplication.** The phone's address must be edited in two files and the camera id must match in two places. They currently agree. A mismatch would make the Cameras page show the node offline or with zero detections while detection itself still works.
- **R-14 Hard-coded backend address in the health page.** The "FastAPI backend" service row shows the literal text `http://127.0.0.1:8000`.

6.4.4 What was not tested

A real phone, real network, real browser, video decoding, the full path from a real vehicle to a real screen, several officers at once, a slow client, several server processes, long-running connections, reconnection after a server restart, and the start-up script's behaviour on another machine.

6.5 User Evaluation

None carried out. No officer used the alert channel, and no survey or interview data exist.

6.6 Analysis of Results

- **The push channel works as designed.** A matching detection reaches every connected police and admin client, unauthorised connections are refused with the standard policy-violation code, and one dead connection does not block the rest.
- **The weaknesses are mostly about the connection's lifetime.** R-2 (token not rechecked), R-8 (no reconnection) and R-3 (disconnect handling) all concern what happens *after* the connection exists. The design assumes a connection is opened once and stays valid. Real networks and real accounts do not behave that way.
- **The monitoring pages are more reassuring than accurate.** R-4 shows that parts of "system health" are constants. A health page that says "operational" without measuring is worse than none for an operator.
- **The proxy solves the address-hiding problem but not the scaling one (R-12).**
- **The 17 ms figure is not a result about the system.** It only shows that the alert channel itself adds negligible delay compared with the seconds a camera pipeline needs.

6.7 Objective-by-Objective Evaluation

Objective	Evidence	Verdict
R1 Push detections to connected clients	TC-R1 to TC-R3, TC-R7	Achieved (in process)
R2 Restrict to police and admin	TC-R4 to TC-R6	Achieved at connect ; a deactivated user still connects (R-2)
R3 Camera status and statistics	TC-R11 to TC-R13	Achieved ; <code>last_ping_seconds</code> missing (R-5)
R4 Feed proxy hiding the camera address	TC-R8 to TC-R10	Achieved for the tested case; token in URL (R-7); scaling untested
R5 System health endpoint	Endpoint returns metrics	Partly achieved : open to the public (R-1) and partly fixed values (R-4)
R6 Integration through contracts and start-up script	Contracts in §4.8; script exists and was read; camera ids and addresses agree	Achieved on the development machine ; the script was not run on another machine and configuration is duplicated (R-13)
R7 Verify and record weaknesses	13 tests, 7 findings from probes, 7 from code reading	Achieved for this scope

6.8 Individual Results

All results here concern the real-time and integration component. The backend test suite, the camera node results and the front-end pages are in the other members' reports and are not claimed.

6.9 Chapter Summary

The alert channel, feed proxy and status logic pass all 13 in-process tests. Probes and code reading found fourteen weaknesses; the most serious are that the browser never reconnects, that a deactivated user's token still opens the alert channel, and that the health endpoint is public and partly fixed values. None has been fixed in this project.

Chapter 7: Discussion

7.1 Introduction

This chapter interprets the results of Chapter 6, compares them with related practice, states the contributions, and discusses limitations.

7.2 Interpretation of Findings

Research question 1 (can a plain WebSocket channel, authenticated with the same token as the rest of the API, deliver detections to all connected officers with no polling?). Yes, in the conditions tested. A match produced a message to every connected police and admin client, unauthorised connections were refused, and a dead connection did not block delivery. The token is reused as designed; the price is that it is checked only once.

Research question 2 (can three processes be integrated using only network contracts?). Yes. The camera node, backend and browser share URLs, message shapes and configuration keys, and none imports another's code. The cost of this loose coupling is visible in R-13: configuration that must agree in several places, with nothing checking that it does.

Research question 3 (what breaks?). The weaknesses fall into three groups:

1. **Lifetime problems** (R-2, R-3, R-8, R-9): the design treats a connection as permanent and a token as permanently valid. Both assumptions fail in real use, and the most damaging consequence is silent: an officer whose connection dropped sees nothing wrong and receives nothing.
2. **Honesty of the monitoring** (R-1, R-4, R-5, R-14): the health and status pages show some values that are measured and others that are constants or mislabelled, and the endpoint is public.
3. **Scaling and resource use** (R-6, R-11, R-12): the design is right for one server process, one camera and a few viewers, and unmeasured beyond that.

The alert channel is not the slow part. The 17 ms in-process figure only shows that broadcasting adds almost no delay. The time an officer waits is dominated by earlier steps: the camera node samples every fifth frame, runs detection and OCR, and is suppressed by a 15-second duplicate window. Improving the alert channel would not speed up a detection much; making it reliable (reconnection, re-checking the user) would matter more.

7.3 Comparison with Existing Work

No numeric comparison was made. Against the approaches in Chapter 2, this component gives push delivery without a broker, and lacks what libraries and brokers provide: automatic reconnection and heartbeats (Socket.IO-style libraries) and fan-out across processes (a broker). The lack of reconnection is the difference that would matter most to an officer. Against monitoring tools, the health page provides a small, readable summary but not measured, authenticated or historical monitoring.

7.4 Contributions of the Project

- **Practical:** a working push channel from detection to police screens on any page, an authenticated camera feed and a status page, and a single-command start-up.
- **Technical:** an alert channel reusing the REST token; per-connection error isolation in broadcast; a way to test the WebSocket route and the feed proxy without a phone, using a local stand-in camera.
- **Analytical:** a documented list of failure modes, separated into those demonstrated in tests and those found only by reading code.

7.5 Limitations and Implications

Limitation	Implication
Everything ran in one process	Timing and connection behaviour on real networks are unknown
Front-end findings are from reading	R-8 to R-10 may behave differently in a browser (for example browsers may retry in some conditions), and should be confirmed by running the page
Stand-in camera, not the phone	Real MJPEG behaviour (frame sizes, stalls, disconnects) was not tested
No live STOLEN alert recorded	The whole path from a real vehicle to a real screen has not been observed
Severity ratings are the student's own	Not a formal scale
Findings not fixed	The delivered system still has them
Probes written by the author	They test the failure modes the author thought of

7.6 Chapter Summary

The alert channel works and is simple. Its limits are about time and scale: what happens when a connection or a token outlives the conditions it was created in, and what happens beyond one process. Those limits, and the parts of the monitoring that are constants, are documented so they can be fixed.

Chapter 8: Conclusion and Recommendations

8.1 Introduction

This chapter summarises the project, states the conclusions the evidence supports, reviews the objectives, and gives recommendations and future work.

8.2 Summary of the Project

The group built SVDS so that a stolen-vehicle report filed by the public can be matched against plates seen by a camera. This report covered the real-time and integration component: a WebSocket channel that pushes a detection to every connected police and admin client, an authenticated proxy for the camera's live feed, camera status and system-health endpoints, the browser's alert connection and toast, and the configuration and start-up script that make the three programs work together. It was verified with in-process WebSocket tests, a stand-in camera and failure-mode probes.

8.3 Conclusions

1. A plain WebSocket channel using the same token as the REST API delivers detections to all connected officers without polling, and refuses reportees and invalid tokens.
2. A relayed MJPEG feed can hide the camera's address and be limited to police and admin.
3. The three programs integrate through network contracts only.
4. The design has confirmed and likely weaknesses: the browser never reconnects, a deactivated user's token still opens the alert channel, the health endpoint is public and partly made of constants, and several resource and scaling limits are untested.
5. No end-to-end latency and no live STOLEN alert were recorded, so the claim that the system alerts officers "in real time" rests on design and in-process tests, not on a measured real-world run.

8.4 Achievement of Objectives

Objective	Status
R1 Push detections	Achieved (in process)
R2 Restrict to police and admin	Achieved at connect; deactivated users still connect

Objective	Status
R3 Camera status	Achieved; one field missing
R4 Feed proxy	Achieved for the tested case
R5 Health endpoint	Partly achieved
R6 Integration and start-up	Achieved on the development machine
R7 Verification and honest record	Achieved for this scope

Research questions 1 and 2: yes. Research question 3: fourteen weaknesses recorded.

8.5 Recommendations

In priority order:

1. **Reconnect automatically in the browser** with a growing delay, show a visible "alerts disconnected" indicator, re-fetch the unread count and recent alerts on reconnect, and clear the ping timer on close (R-8, R-9).
2. **Recheck the user** when a WebSocket connects and periodically after, and close connections when the token expires or the user is deactivated (R-2).
3. **Require a login (admin) for /system/health**, and make each reported value measured: real service checks, real session counts or a renamed label, the actual number of camera nodes (R-1, R-4).
4. **Fix the status field mismatch** by adding a last-seen value to the camera response or changing the page (R-5), and remove the hard-coded address (R-14).
5. **Make disconnect safe** if the connection is already gone (R-3).
6. **Move the tokens out of URLs**: send the WebSocket token as the first message, and use a short-lived, single-purpose feed token or a cookie (R-7).
7. **Cache the camera status** for a few seconds instead of connecting on every request (R-6).
8. **Use one configuration source** for the phone address and camera id, or have the node register itself with the backend (R-13).
9. **If more than one server process or many viewers are expected**, use a message broker for alerts and one shared upstream connection for the feed (R-11, R-12).
10. Show more than one notification, with automatic dismissal (R-10).

8.6 Future Work

- Record a full live run from a real vehicle to a real dashboard, with timestamps at each step, and report the latency.
- Confirm the browser findings by running the pages, and add browser-level tests.
- Add push notifications outside the browser (for example SMS or a mobile app) for officers away from a screen.
- Support several camera nodes with per-node status.
- Add historical monitoring of the camera and the server.

References

1. Fette, I., Melnikov, A. (2011). The WebSocket Protocol. RFC 6455, IETF.
<https://datatracker.ietf.org/doc/html/rfc6455>
2. FastAPI. "WebSockets." FastAPI Documentation.
<https://fastapi.tiangolo.com/advanced/websockets/>
3. Starlette. "WebSockets" and "Testclient." <https://www.starlette.io>
4. MDN Web Docs. "WebSocket API" and "Server-sent events." Mozilla.
<https://developer.mozilla.org>
5. Jones, M., Bradley, J., Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519, IETF.
<https://datatracker.ietf.org/doc/html/rfc7519>
6. OWASP Foundation. "OWASP API Security Top 10." <https://owasp.org>
7. Requests. "Requests: HTTP for Humans." <https://requests.readthedocs.io>
8. IP Webcam (Android application) documentation, describing the MJPEG /video stream.
9. React. "React Documentation." <https://react.dev>
10. Uvicorn. "Uvicorn Documentation." <https://www.uvicorn.org>
11. pytest. "pytest Documentation." <https://docs.pytest.org>
12. Sommerville, I. (2011). Software Engineering, 9th ed. Pearson.

[Add any interviews, Zambia Police Service correspondence or supervisor notes you used, and format all entries to the department's required style.]

Appendix A: Individual Contribution Statement (Group Projects)

Purpose of this Statement

This statement identifies the work for which the student was primarily responsible. It is consistent with the main report. Work by other group members is acknowledged and is not claimed.

A.1 Project and Group Information

Item	Details
Project Title	SVDS: A Camera-Based Stolen Vehicle Detection System for the Zambia Police Service (built on the VRRS registry)
Student Name	[STUDENT NAME]
Computer Number	[COMPUTER NUMBER]
Supervisor	[SUPERVISOR NAME]
Group Size	4
Group Members	1. Moomba Nicholas Katapazi, [COMPUTER NUMBER] (AI/vision) 2. [NAME], [COMPUTER NUMBER] (Backend and security) 3. [STUDENT NAME], [COMPUTER NUMBER] (Real-time and integration) 4. [NAME], [COMPUTER NUMBER] (Frontend, analytics, testing and documentation)

A.2 Overall Group Project

The group built **SVDS/VRRS**, a system with which the public files stolen-vehicle reports and police officers are alerted automatically when a camera sees a reported vehicle. It has three independently running processes: a FastAPI and PostgreSQL backend, a React frontend with three role-based portals (Reportee, Police, Admin), and a camera node that reads number plates from a phone's video stream using a custom-trained YOLOv8 detector and OCR. The common problem is that stolen-vehicle recovery depends on officers chancing to recognise a plate. The core flow is: the camera node reads a plate, the backend matches it against active reports, and connected police dashboards receive an alert over WebSocket.

A.3 Individual Role and Responsibilities

The group's work was divided into four roles. The student's role was **Role 3: Real-time and integration**.

No.	Area of Responsibility	Description
5	Real-time alerts and monitoring	WebSocket alerts pushed to police and admin screens; camera status and live-feed proxy; system-health endpoint; the browser's alert connection and toast
-	Phone-to-server integration	Contracts and configuration tying phone, camera node, backend and front end together; start-up script; running the live demonstration

Other roles, for reference: **Role 1** AI/vision; **Role 2** Backend and security; **Role 4** Frontend, analytics, testing and documentation.

A.4 Detailed Individual Contribution

- **Real-time channel:** wrote the WebSocket connection manager, route and handshake check, and the alert message.
- **Monitoring:** wrote the camera-status, live-feed proxy and system-health endpoints.
- **Browser side:** wired the alert connection, unread count and toast notification into the front end.
- **Integration:** wrote the configuration contract (phone address, camera id, backend address) and `run-all.ps1`.
- **Testing and review:** wrote and ran the in-process WebSocket tests, the stand-in camera tests and the failure-mode probes, and reviewed the browser code, recording fourteen findings.
- **Documentation:** this report.

A.5 Individual Deliverables

No.	Deliverable	Student's Contribution
1	WebSocket alert channel (ConnectionManager, /alerts/ws)	Designed and implemented
2	Camera status, live-feed proxy and health endpoints (routers/system.py)	Designed and implemented

No.	Deliverable	Student's Contribution
3	Browser alert connection and toast (AlertsContext.jsx, NotificationToast.jsx, createAlertSocket)	Designed and implemented
4	Start-up script and configuration contract (run-all.ps1, .env keys)	Designed and implemented
5	Failure-mode review and findings R-1 to R-14	Carried out and documented

A.6 Contribution to the Main System or Research Output

The student produced the part that carries a detection from the backend to the officer's screen and lets operators see the camera and system state. The matching logic and the check-plate endpoint were built by the backend member, who calls the broadcast written here. The camera node was built by the AI/vision member; the page layouts and analytics by the frontend member. The `/system/audit` listing route sits in the same file as the monitoring routes; the records it lists were designed by the backend member.

A.7 Individual Implementation and Technical Work

Modules implemented by the student: the `ConnectionManager` and `/alerts/ws` route in `routers/alerts.py`; the camera-nodes, live-feed and health routes and `is_camera_online` in `routers/system.py`; `AlertsContext.jsx`, `NotificationToast.jsx` and `createAlertSocket` in the front end; `run-all.ps1`.

A.8 Individual Testing and Evaluation Contribution

The student wrote and ran the temporary tests and probes described in Chapter 6 and interpreted their results. The 31-test backend suite includes alert tests, and **who wrote each of those test files should be confirmed by the student**; the report does not claim them.

A.9 Collaboration with Other Group Members

- The **AI/vision member's** camera node sends readings to check-plate; the camera id it sends must equal the backend's `CAMERA_NODE_ID`, and both share the phone stream address.

- The **backend member's** token functions and check-plate endpoint are reused and called.
- The **frontend member's** pages display the alerts, camera and health data, and use the message shape defined here.

A.10 Contribution Summary

1. **Work primarily completed by the student:** the WebSocket channel, camera status, feed proxy, health endpoint, browser alert connection and toast, start-up script and configuration contract.
2. **Work completed jointly with other members:** the alert message shape (with the frontend member); the camera-node contract (with the AI/vision and backend members).
3. **Work primarily completed by other members and used here:** token handling and check-plate matching; the detection model and camera node; page layouts and analytics.

A.11 Estimated Contribution

The percentages below are the student's own estimate and must be completed by the student. They were not derived from any record.

Project Activity	Individual Contribution	Collaborative Contribution
Problem Analysis	[%]	[%]
Literature Review	[%]	[%]
Requirements Analysis	[%]	[%]
System/Research Design	[%]	[%]
Implementation / Development	[%]	[%]
Testing and Evaluation	[%]	[%]
Documentation	[%]	[%]
Other	[%]	[%]

A.12 Statement of Authorship and Contribution

I confirm that the contribution described in this statement accurately represents the work that I personally undertook as part of the group project. I have identified collaborative work and

have not knowingly claimed the work of another group member as my own.

Student Name: ____ ***Computer Number:*** ____ **Signature:** ____ ***Date:*** ____

A.13 Supervisor Verification

I confirm that, to the best of my knowledge, the contribution described above is consistent with the student's participation in the group project.

Supervisor Name: ____ ***Signature:*** ____ **Date:** ____

Appendix B: User Manual / Installation Guide

This guide covers running the three programs together and the real-time features. Backend-only and camera-node-only set-up are in the other members' reports.

B.1 Requirements

- Windows with PowerShell; Python environments for the backend and for the GPU camera node; Node.js for the front end; PostgreSQL.
- An Android phone with an IP Webcam app, on the same network or hotspot as the PC.

B.2 Configure (four places must agree)

File	Key	Value
vrrs-node/.env	PHONE_STREAM_URL	http://<phone-ip>:8080/video
vrrs-backend/.env	CAMERA_STREAM_URL	The same address
vrrs-node/.env	CAMERA_ID	e.g. PHONE-NODE-1
vrrs-backend/.env	CAMERA_NODE_ID	The same id (and optionally CAMERA_NODE_LOCATION)
vrrs-node/.env	BACKEND_URL	http://127.0.0.1:8000
vrrs-frontend/.env	VITE_API_BASE_URL, VITE_WS_URL	http://localhost:8000, ws://localhost:8000

The phone's address changes whenever it reconnects to the hotspot, so update the first two lines each session.

B.3 Start everything

1. On the phone, open IP Webcam and tap **Start server**. Note the address it shows.
2. Update the .env files above.
3. From the project root run:

```
powershell -ExecutionPolicy Bypass -File .\run-all.ps1
```

Three windows open: backend (port 8000), front end (port 5173) and the camera node. The node needs about 20 to 30 seconds to load its models. Close the windows to stop.

B.4 Using the real-time features

- **Alerts:** log in as police or admin. A "Vehicle detected" toast appears on any page when a stolen vehicle is detected, and the unread count in the navigation increases.
- **Cameras page:** shows the live feed and the node's online or offline status. If the feed cannot load it shows "Camera feed unavailable".
- **System Health page (admin):** shows response time, database connections, disk usage and active users.

B.5 Troubleshooting

Symptom	Likely cause	Action
No toast although the node reports STOLEN	The browser's alert connection dropped (the page does not reconnect)	Reload the page; check the browser console for [WS] Disconnected
Cameras page shows offline but detection works	CAMERA_STREAM_URL in the backend .env differs from the node's address	Make them match and restart the backend
Detection count is 0 on the Cameras page	Camera ids differ between node and backend	Make CAMERA_ID and CAMERA_NODE_ID equal
Feed says unavailable	Phone app not running, wrong network, or wrong address	Check the phone app and address
Camera node exits at start	It cannot open the stream	Check PHONE_STREAM_URL
Health page slow to fill	The camera check waits up to 3 s	Wait; it is not an error

B.6 Manual check of the alert channel (no camera needed)

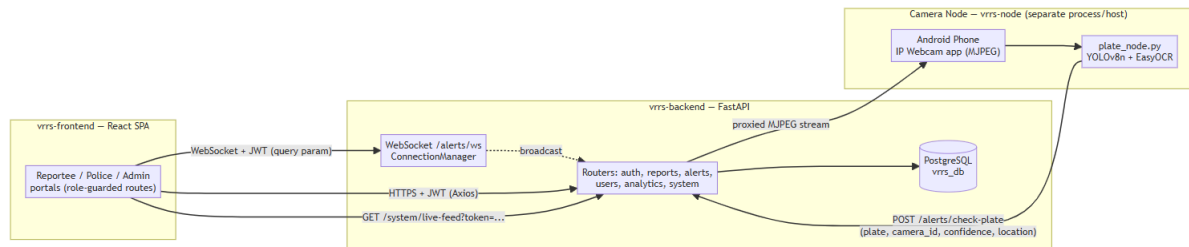
With the backend running and an **active** report for plate TEST123 (a police user activates it), send:

```
curl -X POST http://127.0.0.1:8000/alerts/check-plate -H "Content-Type: application/json" \
  -d '{"license_plate":"TEST123","camera_id":"PHONE-NODE-1","confidence_score":90,"location_spotted":"Test"}'
```

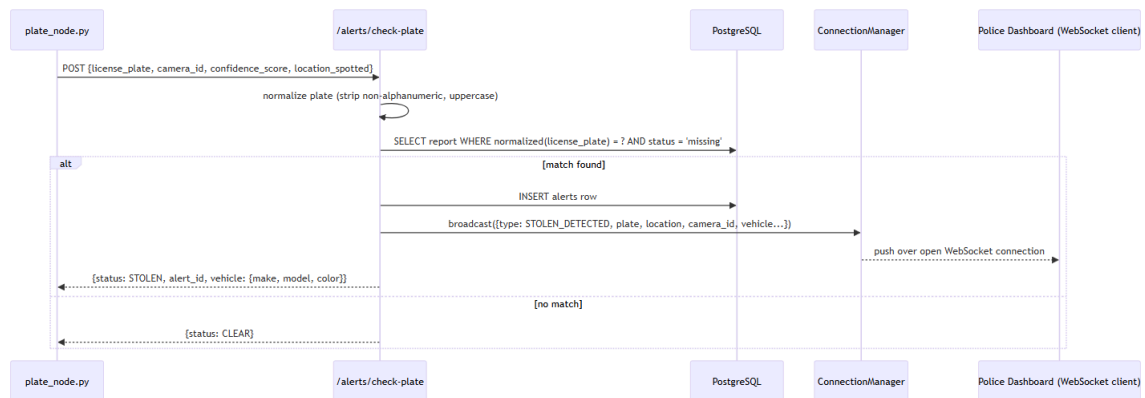
A police browser session should show the toast. Delete the test alert and report afterwards. (This is the manual counterpart of the tests in Chapter 6; it was not run for this report.)

Appendix C: Additional System Designs / Diagrams

C.1 System architecture



C.2 Detection-to-alert sequence



C.3 Screens that use this component (captured earlier from the running system)

SVDS

Police Portal

Vehicle registry

Live alerts

Cameras

Analytics

VR

Zambia Police

Police Portal

Logout

Cameras

Live feed and connected camera nodes

Active cameras

0/1

Live

Offline nodes

1

Needs attention

Live camera feed

Camera nodes

Node ID	Location
PHONE-NODE-1	Live phone camera offline

SVDS

Admin Portal

Overview

User management

System health

Audit log

VR

Zambia Police

Admin Portal

Logout

System health

Monitor service status, camera nodes, and platform performance

API response time

0.3ms

Normal

DB connections

1/15

Healthy

Disk usage

68.2%

151.4 GB free

Active sessions

21

Right now

Core services

FastAPI backend

http://127.0.0.1:8000

operational

PostgreSQL database

Database reachable

operational

Camera nodes

PHONE-NODE-1 - Live phone camera

Offline

offline

SVDS

Police Portal

Vehicle registry

🔔 Live alerts

📷 Cameras

📊 Analytics

VR

Zambia Police

Police Portal

Logout

Live alerts

Real-time camera detections - updates via WebSocket

Unread alerts

0

Live

Alerts today

0

Across 0 nodes

Alert feed

Show 25 ▾

Unread

History

No unread alerts.

Appendix D: Additional Test Cases and Results

D.1 Probe output (raw)

The temporary tests were deleted after running; their printed output is reproduced. (The single "ConnectionResetError" line that appeared in the run came from the stand-in camera closing its socket after sending three frames.)

```
PROBE ws broadcast: STOLEN_DETECTED RT12345 Toyota | http: STOLEN | in-process
ms: 16.9
PROBE ws ping -> pong
PROBE two clients both got: STOLEN_DETECTED STOLEN_DETECTED
PROBE ws reportee token -> closed code 1008
PROBE ws no token -> closed code 1008
PROBE ws garbage token -> closed code 1008
PROBE ws with deactivated user's token -> pong
PROBE double disconnect -> ValueError: list.remove(x): x not in list
PROBE broadcast with dead conn: remaining = 1 | live got: {"x": 1}
PROBE /system/health no auth -> 200 ['api_response_ms', 'db_connections_used',
'db_connections_max', 'disk_usage_percent']
PROBE health services: [('FastAPI backend','operational'), ('PostgreSQL
database','operational'), ('WebSocket alerts','operational'), ('JWT auth
service','operational')]
PROBE camera-nodes keys:
['avg_confidence','camera_id','detections','last_seen','location','status'] |
has last_ping_seconds: False | status: offline
PROBE health active_sessions = 0
PROBE live-feed police -> 200 multipart/x-mixed-replace; bou | bytes: 132 | has
frames: True
PROBE live-feed reportee -> 401 | no token -> 422
PROBE camera status with reachable stream -> online
PROBE health camera online -> 1
PROBE live-feed unreachable -> 503 in 2.0s
PROBE camera status unreachable -> offline in 2.1s
```

After the probes were removed, the existing suite was run again: 31 passed.

D.2 One test as an example

```
def test_ws_receives_broadcast(client, make_user, auth_headers, db_session):
    _seed(db_session, make_user)                # an active report for
    plate RT12345
```

```
police = make_user(role="police", badge="B1")
token = auth_headers(police)["Authorization"].split()[1]
with client.websocket_connect(f"/alerts/ws?token={token}") as ws:
    resp = client.post("/alerts/check-plate", json={"license_plate":
"RT12345", ...}).json()
    msg = ws.receive_json()
    assert msg["type"] == "STOLEN_DETECTED" and resp["status"] == "STOLEN"
```

Appendix E: Data Collection Instruments

Not applicable. No questionnaires, interviews or surveys were used.

Appendix F: Additional Code / Configuration

F.1 Browser socket helper (vrrs-frontend/src/services/api.js)

```
export const createAlertSocket = (onMessage) => {
  const token = localStorage.getItem("vrrs_token");
  const ws = new WebSocket(`${WS_BASE_URL}/alerts/ws?token=${token}`);
  ws.onopen = () => { setInterval(() => ws.readyState === 1 &&
ws.send("ping"), 30000); };
  ws.onmessage = (e) => onMessage(JSON.parse(e.data));
  ws.onclose = () => console.log("[WS] Disconnected");
  ws.onerror = (e) => console.error("[WS] Error:", e);
  return ws;
};
```

Findings R-8 (no reconnection) and R-9 (the timer is never cleared) come from this function.

F.2 Camera online check (vrrs-backend/app/routers/system.py)

```
def is_camera_online() -> bool:
    if not CAMERA_STREAM_URL:
        return False
    try:
        requests.get(CAMERA_STREAM_URL, timeout=3, stream=True).close()
        return True
    except requests.RequestException:
        return False
```

F.3 Start-up script (run-all.ps1, abridged)

```
Start-Process powershell -ArgumentList "-NoExit","-Command","cd '$root\vrrs-backend'; .\env\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000"
Start-Process powershell -ArgumentList "-NoExit","-Command","cd '$root\vrrs-frontend'; npm run dev"
Start-Process powershell -ArgumentList "-NoExit","-Command","cd '$root\vrrs-node'; C:\Users\user\anaconda3\envs\yolov8-env\python.exe plate_node.py"
```

The camera-node line contains an absolute path to the developer's own Python environment, so the script works only on that machine until the path is changed.

Appendix G: Other Supporting Material

G.1 Group role split

Role	Work areas	Focus
1. AI/vision	Detection model; camera node	YOLOv8n detector, OCR pipeline
2. Backend and security	Backend API and database; security and access control	FastAPI, PostgreSQL, JWT, roles
3. Real-time and integration	Real-time alerts and monitoring; phone-to-server wiring	This report
4. Frontend	Frontend portals; analytics, testing and documentation	React pages, analytics, run scripts

G.2 Findings summary

ID	Finding	Basis	Severity
R-1	Health endpoint needs no login	Probe	Medium
R-2	Deactivated user's token opens the alert channel	Probe	High
R-3	<code>disconnect()</code> raises on double removal	Probe	Low
R-4	Health values partly fixed or misnamed	Probe and code	Medium
R-5	<code>last_ping_seconds</code> missing from camera response	Probe and code	Low
R-6	Camera check blocks up to 3 s	Probe	Low
R-7	Tokens in URLs	Code	Medium
R-8	Browser never reconnects	Code	High
R-9	Ping timer never cleared	Code	Low
R-10	One notification slot, no auto-dismiss	Code	Low
R-11	In-memory connection list, one process	Code	Design limit
R-12	Feed proxy resource use	Code	Medium
R-13	Duplicated configuration	Code and check	Medium
R-14	Hard-coded backend address in health page	Code	Low